

PROJECT PLAN

AIFX Phase 2 Face Swap API - GCP Deployment Plan

Four-day implementation plan for a public, API-account-controlled cloud service

Prepared for: Zooey / AIFX Studio Phase 2

Prepared on: July 6, 2026

Execution window: 4 working days

Deployment target: Google Cloud Platform: Cloud Run, Cloud Storage, Firestore, Secret Manager

Out of scope: No separate buffer-task plan in this document

Executive Summary

The recommended Phase 2 path is to deploy the face swap API on GCP so external testers, ComfyUI, or AIFX Studio can call one stable HTTPS endpoint protected by API accounts. The service owns face detection, crop handling, LoRA mapping, ComfyUI API orchestration, image merging, cloud storage, task history, and logs. ComfyUI itself is assumed to already be available and will be called as an external service.

1. Target Outcome

Deliver a GCP-hosted FastAPI service that exposes a production-shaped Phase 2 endpoint:

Primary API

POST /api/v1/face-swap

Authorization: Bearer <API_ACCOUNT_KEY>

- Accept image uploads through a public HTTPS API.
- Authenticate external callers with API accounts and scoped API keys.
- Detect and crop faces using the existing Phase 1 pipeline, with future room for MediaPipe + InsightFace fusion.
- Map each face or character assignment to the correct LoRA filename.
- Call the existing external ComfyUI API workflow.
- Blend the returned face output back into the original image without visible square crop edges.
- Store original, cropped, ComfyUI output, and final images in Cloud Storage.
- Store API account data, task history, LoRA config, and request logs in Firestore.

2. Recommended GCP Architecture

Layer	GCP Service	Purpose
API runtime	Cloud Run	Host the FastAPI face swap backend as a public HTTPS service.
Image storage	Cloud Storage	Persist original images, crops, ComfyUI outputs, and final merged images.
Metadata	Firestore	Store task history, API accounts, request logs, and LoRA config.
Secrets	Secret Manager	Store ComfyUI endpoint/token, API account config, and service secrets.
Logs	Cloud Logging	Inspect request failures, ComfyUI timeouts, and deployment issues.
Access control	IAM	Allow Cloud Run to read secrets and write Storage/Firestore records.

Important Boundary

This plan deploys the face swap API and supporting data layer on GCP. ComfyUI is treated as an already available external service; the API only needs to call it reliably.

3. API Account Plan

API accounts replace a single shared API token. Each external caller can receive its own key, status, scopes, and logs.

Collection	Key Fields	Purpose
api_accounts	account_id, name, status, scopes, rate_limit_per_minute, created_at	Represents an external caller such as Henry test account or ComfyUI worker account.
api_keys	account_id, key_prefix, key_hash, status, expires_at, last_used_at	Stores hashed keys only; raw API keys are shown once at creation.
api_request_logs	account_id, endpoint, status_code, latency_ms, error_message, created_at	Provides auditability and debugging for public API access.

- Missing or malformed Authorization header returns 401.
- Invalid API key returns 401.
- Disabled account returns 403.
- Missing face:swap scope returns 403.
- Valid account may call /api/v1/face-swap and receives task_id plus result URLs.

4. Four-Day Implementation Plan

Day	Goal	Main Work	Definition of Done
Day 1 - July 6	Cloud architecture and API contract	Finalize <code>/api/v1/face-swap</code> request/response; design API account collections; create GCP project resources: Cloud Run API, Storage bucket, Firestore, Secret Manager, service account.	GCP resources exist; API contract is frozen; account model and deployment target are documented.
Day 2 - July 7	API account, cloud storage, and task records	Implement API account middleware; add Cloud Storage adapter; add Firestore task history and request logs; build <code>/api/v1/face-swap</code> skeleton; store uploaded original image; run Phase 1 detection and preserve bounding boxes.	Valid API key can create a task; invalid access is rejected; image and task records are stored in GCP services.
Day 3 - July 8	ComfyUI integration and merging	Load ComfyUI API workflow JSON; inject crop image and LoRA filename; call ComfyUI; support one or multiple faces sequentially; store ComfyUI outputs; blend results back into the original image.	Single-face end-to-end flow works; multi-face loop is supported; final merged image is stored and returned.
Day 4 - July 9	Cloud Run deployment and public API testing	Deploy FastAPI to Cloud Run; configure IAM and secrets; run public API tests for health, unauthorized access, valid account, image processing, LoRA missing, and ComfyUI timeout; update README and handoff notes.	Cloud Run URL is live; API account access works; final image URL is returned; README contains deployment and testing instructions.

5. Daily Execution Details

Day 1 - July 6: GCP Foundation

- Create or select the GCP project and enable Cloud Run, Cloud Storage, Firestore, Secret Manager, and Cloud Logging.
- Create the Cloud Storage bucket for original, crop, ComfyUI output, and final images.
- Create Firestore collections for `task_history`, `api_accounts`, `api_keys`, `api_request_logs`, and `lora_config`.
- Define the Cloud Run service account and IAM permissions.
- Freeze the API request/response shape for `/api/v1/face-swap`.

Day 2 - July 7: Auth and Persistence

- Implement API account authentication with hashed API keys and scopes.
- Implement GCP Cloud Storage upload paths grouped by `account_id` and `task_id`.
- Write Firestore task state transitions: `pending`, `detecting`, `cropping`, `comfyui_processing`, `blending`, `completed`, `failed`.
- Keep original-image pixel coordinates from Phase 1 metadata.
- Return `task_id`, face list, bounding boxes, and stored image URLs.

Day 3 - July 8: ComfyUI and Blending

- Extract the ComfyUI workflow in API JSON format and identify dynamic input nodes.
- Inject cropped face image and LoRA filename into the JSON payload.

- Submit to ComfyUI and listen for completion or timeout.
- Resize and blend each returned face into its original coordinates.
- Use alpha feathering first; upgrade to OpenCV seamlessClone if edge artifacts remain.

Day 4 - July 9: Deployment and Review

- Deploy the service to Cloud Run using source deploy first; add Docker only if dependency installation fails.
- Set production environment variables and Secret Manager references.
- Run public curl tests with and without API account keys.
- Verify Cloud Storage output URLs, Firestore task records, and Cloud Logging errors.
- Prepare handoff notes: API base URL, API account key, supported character IDs, and known limitations.

6. Final Deliverables

Deliverable	Required	Notes
Cloud Run FastAPI service	Yes	Public HTTPS API for Phase 2 face swap.
POST /api/v1/face-swap	Yes	Fully automated API entry point.
API account authentication	Yes	API keys with account status and scopes.
Cloud Storage integration	Yes	Original, crop, ComfyUI output, final image.
Firestore task history	Yes	Status tracking, logs, LoRA config.
ComfyUI API wrapper	Yes	Dynamic workflow JSON injection.
Blending module	Yes	No visible square boundaries around merged faces.
README and curl examples	Yes	Reviewer can call the public API.

7. Success Criteria

- A public Cloud Run URL is available.
- Only valid API accounts can call /api/v1/face-swap.
- The API stores images and task records in GCP services.
- The API can call the existing ComfyUI service.
- The final merged image is returned as a URL.
- Failures are recorded without crashing the service.

Minimum Successful Phase 2 Version

A GCP-hosted public API, protected by API account credentials, that accepts an image, calls ComfyUI, stores artifacts, and returns a final merged image URL.